SOUTENANCE DE MÉMOIRE DE FIN D'ÉTUDES

Réduction de la dette technique dans un environnement de production

Master 2 Informatique — parcours Génie Logiciel · deux années d'alternance

PRÉSENTÉ PAR

Jérémy Hurel

TUTEUR UNIVERSITAIRE

Mickaël Montassier — Univ. de Montpellier

TUTEUR ENTREPRISE

David Desmarest — Free · Beemo

MÉMOIRE RÉALISÉ CHEZ

Année universitaire 2024 – 2026

Deux ans, deux entreprises



“

Livrer une première version du code, c'est comme s'endetter. Une petite dette accélère le développement à condition d'être remboursée rapidement par une réécriture.

— Ward Cunningham, *The WyCash Portfolio Management System*, 1992

Un emprunt, pas une faute

La dette technique n'est pas une faute morale : c'est un **emprunt** que l'on contracte, parfois à raison, pour livrer plus vite.

La métaphore financière

Principal : l'écart entre le code tel qu'il est et tel qu'il devrait être.

Intérêts : le surcoût payé à chaque intervention tant que le principal n'est pas remboursé.

Remboursement : il porte un nom, le *refactoring*.

Un arbitrage permanent

La dette n'est presque jamais un problème purement technique : c'est un arbitrage entre

le *time-to-market* attendu par le marché,

et la soutenabilité du logiciel : coût de maintenance, risque de régression, fiabilité.

Trois points qui me tiennent à cœur

01 **Les enjeux sociaux de la dette technique**
Motivation des équipes, dette de connaissance et coût humain du legacy

02 **L'IA, une solution pleine de promesses**
Un levier puissant mais conditionné, et ses propres risques de dette

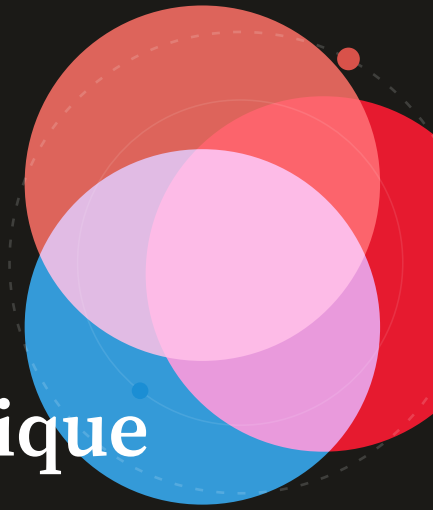
03 **Écrire du code dans un existant fortement endetté**
L'adressage incrémental et l'isolation du code neuf sur une base legacy

01

PARTIE 01

Les enjeux sociaux de la dette technique

La forme de dette la plus dangereuse n’est pas dans le code : elle part avec les personnes.



Université de Montpellier Free Mobile Beemo Technologie

La dette n'est pas qu'un problème de code

Chronophage et ingrat

Reprendre le contrôle d'un logiciel *legacy* est un travail long, et peu gratifiant.

Un coût affectif mesuré

La psychométrie l'observe : le code endetté génère déplaisir, anxiété, jusqu'à la réticence à intervenir.

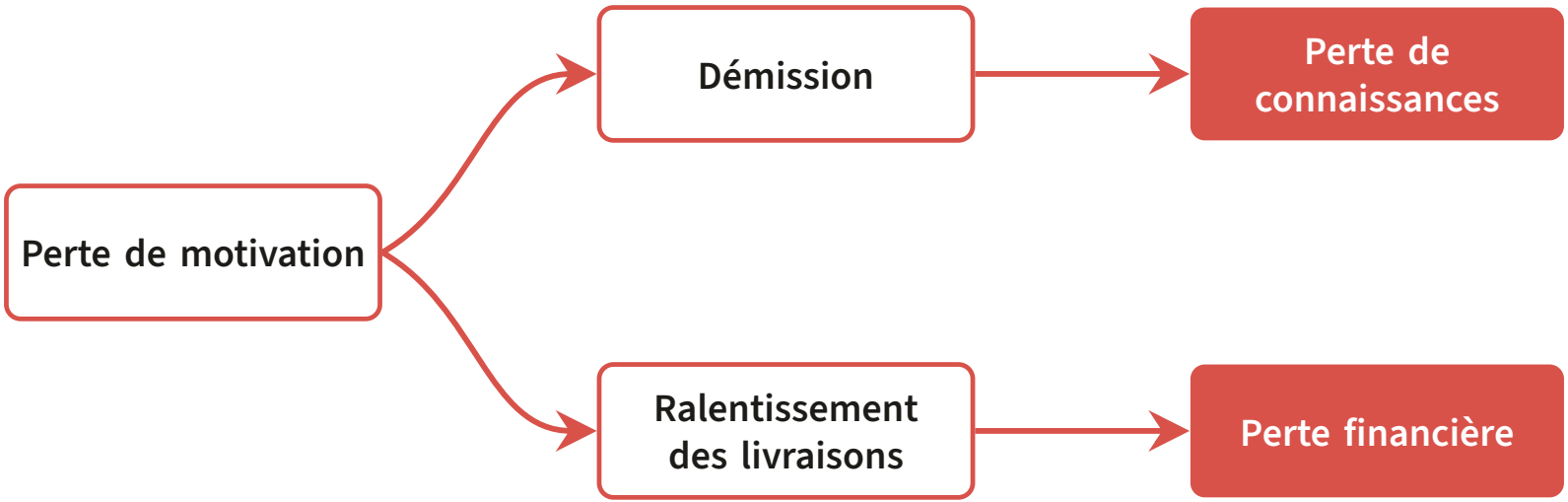
La « dette sociale »

Des structures d'équipe sous-optimales génèrent, comme le code, un surcoût croissant.

La motivation, un levier

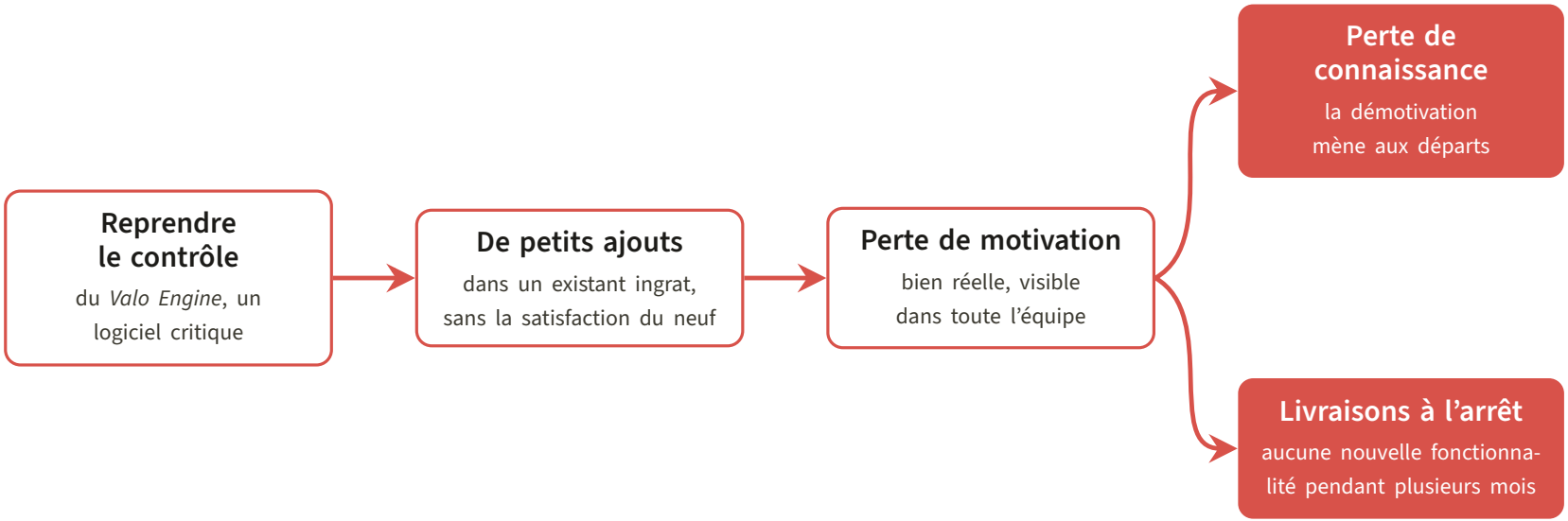
Pour résorber une dette, elle pèse autant que l'outillage ou l'architecture.

Deux spirales, un même déclencheur



Un même déclencheur, la démotivation, alimente deux pertes : la connaissance et l'argent.

Ce que j'ai observé chez Free Mobile



Le terreau de la dette sociale : un enchaînement, pas un accident isolé.

Une cascade de départs

Un ultimatum de six mois
Un développeur pose ses conditions, puis quitte l'équipe.

Le départ de David
Perte d'attrait pour l'environnement et changement de cap stratégique entraînant son départ.

Un collègue change d'équipe
Le sentiment d'avoir « jeté au placard » plus d'un an de développement.

Mon propre départ
Free Mobile en janvier, pour rejoindre Beemo en avril.

Chaque départ emporte une part de la connaissance du logiciel.

Deux leçons sur l'enjeu social

Garder le cap

En changer trop vite démotive et génère de la nouvelle dette : le *quick fix* qui périme un an de travail déjà réalisé.

La dette de connaissance

La plus dangereuse : invisible dans le code, elle ne se révèle qu'au départ d'une personne.

Sa parade

Capitaliser sous forme **exécutable** (tests, patchs versionnés, outillage) plutôt que dans la tête des gens.

02

PARTIE 02

L'IA, une solution pleine de promesses

Un levier dont la productivité est proportionnée à la maîtrise de celui qui le pilote.

Université de Montpellier Free Mobile Beemo Technologie

Un changement de cap, de nouveaux outils

Une politique revue

En révisant sa stratégie, Free Mobile rouvre la porte aux **IA externes**.

Des licences déployées

De l'**IA agentique** (Cursor) mise entre les mains de tous les développeurs.

Une avance personnelle

J'étais déjà **beta-testeur de GitHub Copilot**.

Un terrain d'essai

Le portage du framework devient le banc idéal pour éprouver ces capacités réelles.

Le terrain : le framework de test du Valo Engine

Scénarios déclaratifs
en YAML : on décrit *quoi* vérifier, jamais *comment*

→

Surcouche de Pytest
~30 000 lignes de Python qui exécutent les scénarios

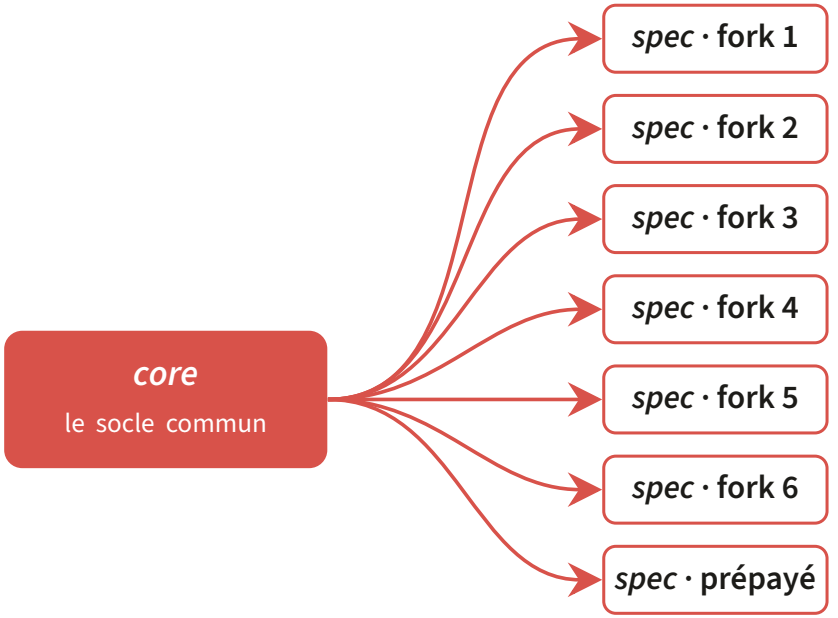
→

Le filet de sécurité
couvre le moteur de non-régression

200+
scénarios de test

75 %
de couverture du moteur

Un refactoring pour le portage, une transmission



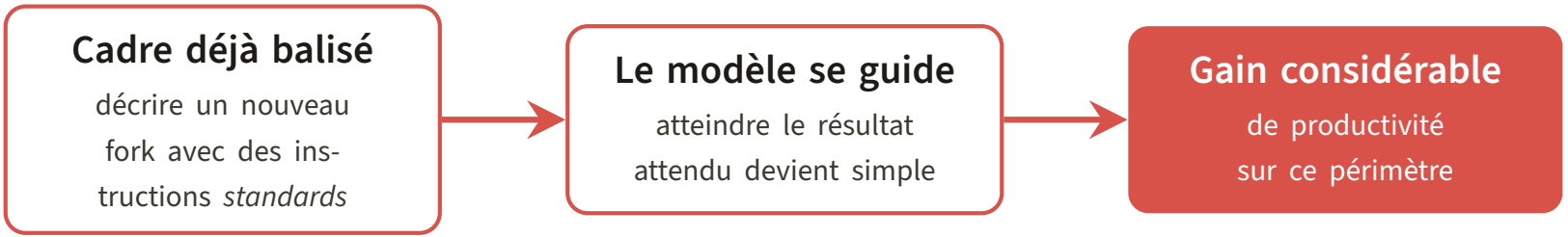
Une refonte *core* / *spec*

Un socle commun, puis une *spec* par fork qui n'isole que les différences métier.

Une transmission assumée

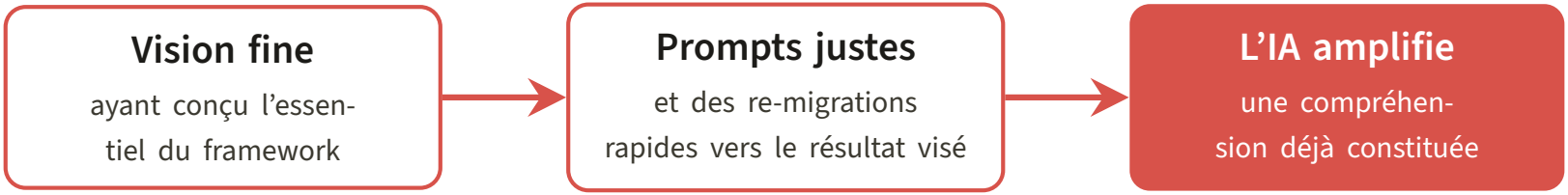
Mené en **binôme**, pour transmettre la connaissance du framework avant mon départ.

Sur un cadre balisé : un gain considérable



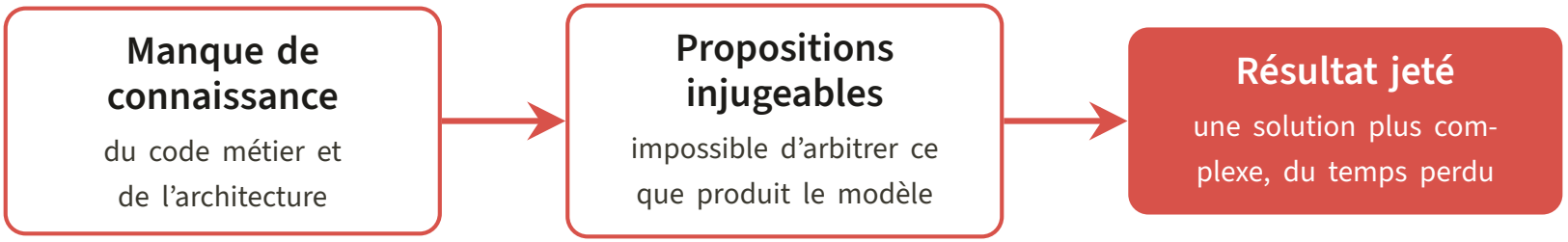
Un constat qui rejoint fidèlement les conclusions d'une **étude d'IBM** sur l'impact des LLM dans la production de code.

Facteur 1 : la maîtrise de l'architecture



L'IA amplifie la compréhension ; elle ne s'y substitue pas.

Facteur 2, *a contrario* : sans la connaissance



Cas vécu : le décodage des CDR, mené en mon absence avec une autre solution agentique.

Le bottleneck : les revues de code

Des volumes massifs

L'IA produit de grands pans de code d'un seul tenant, ardus à relire.

Une équipe sous tension

De nouveaux qui poussent des *merge requests*, des historiques sous forte charge mentale.

Le retour de la dette sociale

Par un autre chemin que celui de Free Mobile, le même surcoût humain.

Déléguer la revue à l'IA ?

Au prix de la **transmission** et de l'**harmonisation**, deux garde-fous.

Le coût, lui, est bien réel

200 à 500 €

par mois et par développeur

Un abonnement non maîtrisé

Qui s'accumule au fil de l'usage, sans plafond clair.

Plus le coût du développeur

L'IA ne le remplace pas : elle le seconde, et reste à ses côtés.

L'IA : un levier, pas un substitut

La compétence d'abord

Ce qui fait de l'IA un outil puissant, c'est la maîtrise du développeur qui la pilote.

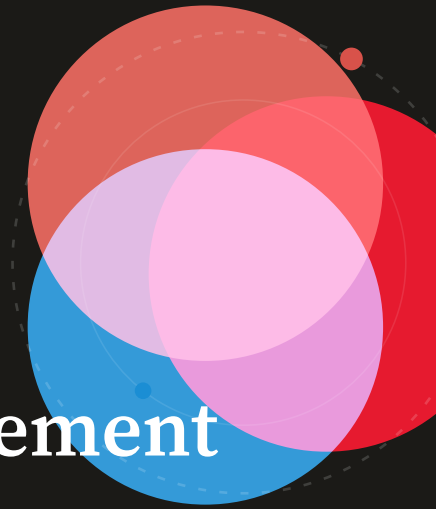
Elle récompense la maîtrise

De l'architecture et du métier ; sur un terrain mal connu, elle produit un code plus complexe.

Son propre risque de dette

Revues alourdies, harmonisation et transmission menacées.

03



PARTIE 03

Écrire du code dans un existant fortement endetté

Ne pas toujours résorber l'existant, mais savoir construire proprement à côté.

Université de Montpellier Free Mobile Beemo Technologie

La solution la plus simple : l'incrémental

Geler est illusoire

Le marché du numérique est trop volatil pour s'arrêter de livrer.

Réécrire à l'occasion

Partir de l'existant et refondre le code concerné quand une nouvelle fonctionnalité le traverse.

La même logique partout

Observée chez **Free Mobile** comme chez **Beemo**.

Pas un grand soir

Mais une accumulation d'**actions ciblées** qui remboursent la dette.

Le code chez Beemo : un firmware très endetté

247 k
lignes de code

0,0 %
de couverture de tests

33 312
de complexité cyclomatique
dont **7 373** pour `Dedup.cpp`

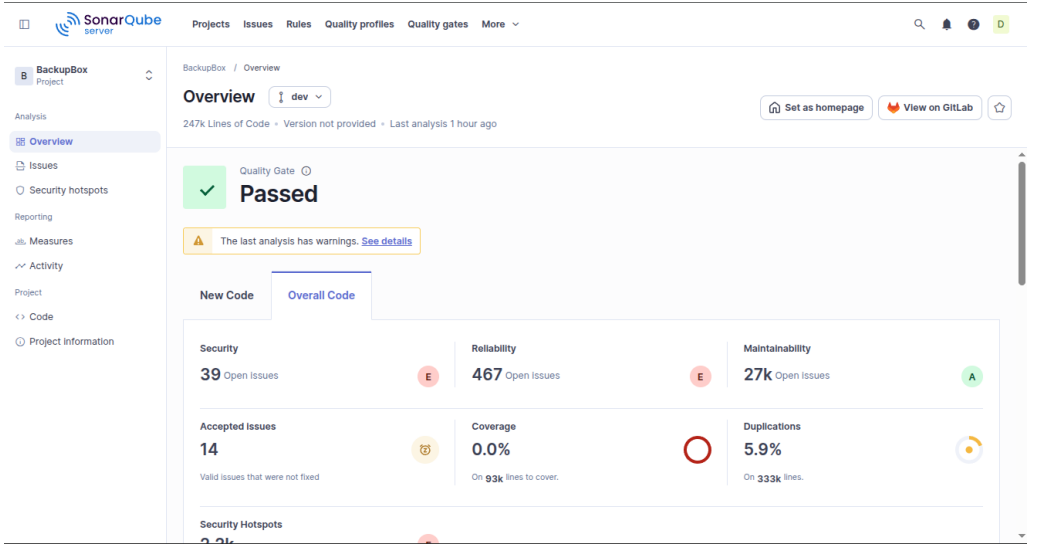
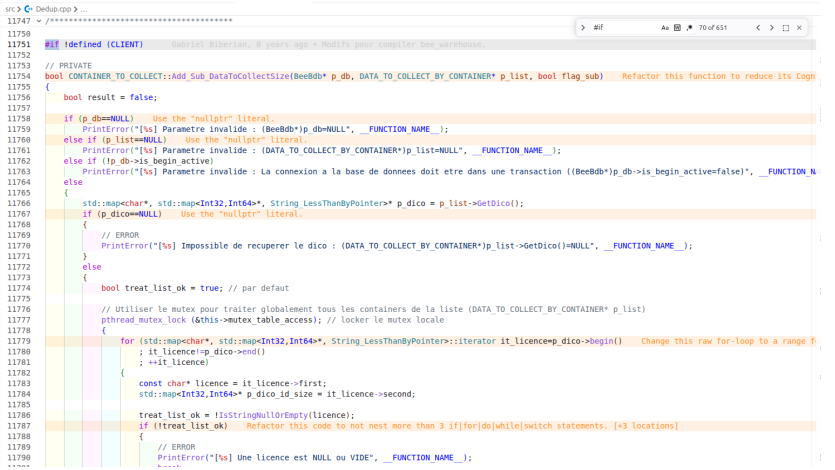


Tableau de bord SonarQube du firmware de la Beebox.

Les symptômes, un par un

- Un **abus de directives de préprocesseur** (`#ifdef`) pour gérer les cas de compilation.
- Des **fonctions interminables** et une maîtrise approximative du C++ (jusqu'à des tests `this` == `nullptr`, un *undefined behavior*).
- Une **absence totale de tests** : aucun filet de sécurité.
- Un `main.h` qui **inclut tout le projet** : un couplage massif.



```
src > Dedup.cpp > ...
11747 /****************************************
11750
11751 #if !defined (CLIENT)    Gabriel Biberian, 8 years ago • Modifs pour compiler bee_warehouse.
11752
11753 // PRIVATE
11754 bool CONTAINER_TO_COLLECT::Add_Sub_DataToCollectSize(BeeBdb* p_db, DATA_TO_COLLECT_BY_CONTAINER* p_list, bool flag_sub)    Refactor this function to reduce its Cogn
11755 {
11756     bool result = false;
11757
11758     if (p_db==NULL)    Use the "nullptr" literal.
11759         PrintError("[%s] Parametre invalide : (BeeBdb*)p_db=NULL", __FUNCTION_NAME__);
11760     else if (p_list==NULL)    Use the "nullptr" literal.
11761         PrintError("[%s] Parametre invalide : (DATA_TO_COLLECT_BY_CONTAINER*)p_list=NULL", __FUNCTION_NAME__);
11762     else if (!p_db->is_begin_active)
11763         PrintError("[%s] Parametre invalide : La connexion a la base de donnees doit etre dans une transaction ((BeeBdb*)p_db->is_begin_active=false)", __FUNCTION_N
11764     else
11765     {
11766         std::map<char*, std::map<Int32,Int64>*, String_LessThanByPointer>* p_dico = p_list->GetDico();
11767         if (p_dico==NULL)    Use the "nullptr" literal.
11768         {
11769             // ERROR
11770             PrintError("[%s] Impossible de recuperer le dico : (DATA_TO_COLLECT_BY_CONTAINER*)p_list->GetDico()=NULL", __FUNCTION_NAME__);
11771         }
11772         else
11773         {
11774             bool treat_list_ok = true; // par defaut
11775
11776             // Utiliser le mutex pour traiter globalement tous les containers de la liste (DATA_TO_COLLECT_BY_CONTAINER* p_list)
11777             pthread_mutex_lock (&this->mutex_table_access); // locker le mutex locale
11778             {
11779                 for (std::map<char*, std::map<Int32,Int64>*, String_LessThanByPointer>::iterator it_licence=p_dico->begin()    Change this raw for-loop to a range f
11780                     ; it_licence!=p_dico->end()
11781                     ; ++it_licence)
11782                 {
11783                     const char* licence = it_licence->first;
11784                     std::map<Int32,Int64>* p_dico_id_size = it_licence->second;
11785
11786                     treat_list_ok = !IsStringNullOrEmpty(licence);
11787                     if (!treat_list_ok)    Refactor this code to not nest more than 3 if|for|do|while|switch statements. [+3 locations]
11788                     {
11789                         // ERROR
11790                         PrintError("[%s] Une licence est NULL ou VIDE", __FUNCTION_NAME__);
```

Un extrait du firmware historique.

Le pattern : isoler le neuf du legacy

- Une **couche d'adaptation** (idiome **PIMPL**) joue le rôle de **frontière**.
- Tout le couplage avec le legacy est **confiné à une seule unité de traduction**.
- Le code neuf compile contre une **interface propre**, **testable** et **isolée**.



Le couplage reste confiné dans le *wrapper* ; le neuf n'en dépend jamais directement.

Automatiser un maximum de tâches

Tester avant le *merge*

Le framework s'exécute en CI : un défaut est intercepté avant d'atteindre la production.

Mesurer l'invisible

Le bénéfice se compte en incidents qui n'ont jamais eu lieu, pas en incidents corrigés.

Industrialiser le reste

Dupliquer la supervision (Grafana), versionner les builds, rendre la compilation reproductible.

Construire proprement à côté : l'immuabilité

La fonctionnalité

Rendre un jeu de sauvegarde **insupprimable et inaltérable** pendant un délai de rétention.

Deux briques propres

En **C++ moderne**, isolées et testables : un automate à états et un service de chiffrement.

Sans aggraver la dette

Écrite à côté du firmware qui l'accueille, jamais en l'alourdissant.

L'enjeu métier

Elle ferme un dernier angle d'attaque : la compromission de la box elle-même.

Conclusion

Un arbitrage économique

La dette est permanente, pas seulement un problème de code.

Une stratégie incrémentale

Livrer tout en réduisant le risque des interventions suivantes.

La dette de connaissance

La forme la plus dangereuse : elle part avec les personnes.

L'IA, un levier

Elle récompense la compétence, sans jamais s'y substituer.

Ce que cette alternance m'a appris

Un spectre technique large

C++ critique et embarqué, Python (un framework de 30 000 lignes), TypeScript, protocoles LSP/DAP, rétro-ingénierie.

Une compétence transversale

Reprendre méthodiquement le contrôle d'un système opaque et non documenté.

Deux contextes

Un grand opérateur et une PME, et l'exercice exigeant de **transmettre**.

Merci de votre attention

Je remercie le jury et mes encadrants pour leur accompagnement,
et reste à votre disposition pour vos questions.

CONTACT

jeremyhurel34@gmail.com

STRUCTURE D'ACCUEIL

Free Mobile · Beemo Technologie